

HC-05 speed

HC-05 Bluetooth module communication speed depends on UART baud rate and the real Bluetooth SPP throughput.

HC-05 speed summary

Item	Typical value
Default UART baud rate	9600 bps
Common stable baud rates	38400, 57600, 115200 bps
Often usable max UART baud	230400 / 460800 bps
Possible configured baud	Up to 1,382,400 bps on some modules
Practical reliable app data speed	Usually 10–40 KB/s
Bluetooth type	Bluetooth 2.0 + EDR, SPP serial profile

Important point

Even if you set HC-05 UART to 115200 bps, the real useful data speed is lower because of:

- Bluetooth packet overhead
- serial framing overhead
- phone/PC Bluetooth stack delay
- small buffer size
- retransmission/interference

So for a real mobile ↔ Qt app sync, I would assume:

Safe design speed: 10 KB/s

Good condition speed: 20–40 KB/s

UART baud rate comparison

Baud rate	Raw bits/sec	Approx max bytes/sec
9600	9.6 kbps	~960 bytes/s
38400	38.4 kbps	~3.8 KB/s
57600	57.6 kbps	~5.7 KB/s
115200	115.2 kbps	~11.5 KB/s
230400	230.4 kbps	~23 KB/s
460800	460.8 kbps	~46 KB/s

Because UART uses start/stop bits, 1 byte usually costs 10 bits, so:

```
bytes/sec ≈ baud rate / 10
```

For your inventory app

For sending only:

```
barcode
```

```
quantity
```

```
operation type
```

```
timestamp
```

```
optional note/source/destination
```

HC-05 is enough.

Example message size:

```
{  
  
  "barcode": "6921234567890",  
  
  "qty": 5,  
  
  "type": "ADD"  
  
}
```

This may be only around 50–150 bytes.

At 115200 baud, even with overhead, you can send hundreds of these records per second.

My recommendation

Use:

UART baud rate: 115200

Protocol: small JSON or compact binary/message format

Picture transfer: avoid

Your idea to not transfer product pictures is correct. HC-05 is okay for barcode and quantity syncing, but it is not good for images. For pictures, the phone should keep them locally, and Qt should only receive barcode/product data.

what should be ready for qt-hc05 communication?

Thought for a couple of seconds

For Qt ↔ HC-05 Bluetooth communication, you need 4 things ready:

1. HC-05 hardware setup
2. Microcontroller firmware
3. Qt Bluetooth/Serial communication code
4. A simple sync protocol

For your app, I recommend this structure:

Mobile App / Scanner Device

↓

Microcontroller

↓ UART

HC-05 Bluetooth

↓ Bluetooth SPP

Qt Desktop App

Or, if the phone talks directly to Qt, then HC-05 may not be needed. HC-05 is mainly useful when a microcontroller device sends barcode/stock data to the Qt app.

1. Hardware needed

You need:

Item	Purpose
HC-05 module	Bluetooth serial communication
Microcontroller	Arduino, ESP32, STM32, etc.
Barcode scanner or mobile scanner logic	Reads barcode
Power supply	Usually 3.3V or 5V depending on board
Logic level protection	Important for HC-05 RX pin

Important wiring:

HC-05 TX → Microcontroller RX

HC-05 RX ← Microcontroller TX

HC-05 VCC → 5V or 3.3V depending on module

HC-05 GND → GND

Very important:

Microcontroller TX may be 5V.

HC-05 RX usually expects 3.3V logic.

Use a voltage divider or level shifter for HC-05 RX.

2. HC-05 configuration

You should configure the HC-05 before using it.

Recommended settings:

Mode: Slave

UART baud rate: 115200

Name: StockSync-HC05

PIN: 1234 or custom PIN

Role: 0

Common AT commands:

AT

AT+NAME=StockSync-HC05

```
AT+ROLE=0
```

```
AT+UART=115200,0,0
```

```
AT+PSWD=1234
```

For simple use, slave mode is enough. The Qt desktop app can connect to the HC-05.

3. Qt side requirements

Your Qt app needs Bluetooth Serial Port Profile support.

In Qt, you usually need:

```
#include <QBluetoothDeviceDiscoveryAgent>
```

```
#include <QBluetoothSocket>
```

```
#include <QBluetoothServiceDiscoveryAgent>
```

Qt module:

```
QT += bluetooth
```

If using CMake:

```
find_package(Qt6 REQUIRED COMPONENTS Bluetooth)
```

```
target_link_libraries(your_app PRIVATE Qt6::Bluetooth)
```

Qt app should support:

```
Scan Bluetooth devices
```

```
Show available HC-05 devices
```

```
Pair/connect to selected HC-05
```

```
Open serial Bluetooth socket
```

```
Read incoming data
```

```
Send ACK/NACK response
```

```
Handle disconnect/reconnect
```

4. Operating system readiness

Windows

You usually need to:

```
Pair HC-05 from Windows Bluetooth settings
```

```
Enter PIN
```

```
Windows creates COM port
```

```
Qt can connect using Bluetooth socket or serial COM port
```

For Windows, using QSerialPort over the created COM port may be easier than using Qt Bluetooth directly.

Qt modules:

```
QT += serialport
```

or CMake:

```
find_package(Qt6 REQUIRED COMPONENTS SerialPort)
```

```
target_link_libraries(your_app PRIVATE Qt6::SerialPort)
```

Then the Qt app talks to HC-05 like a normal serial port.

Example:

```
COM5
```

```
115200 baud
```

```
8 data bits
```

```
No parity
```

```
1 stop bit
```

```
No flow control
```

Linux

You may need:

```
sudo apt install bluetooth bluez
```

You can pair using Bluetooth settings or `bluetoothctl`.

After pairing, HC-05 may appear as:

```
/dev/rfcomm0
```

Then Qt can read/write via serial port.

5. Communication protocol

Do not just send random text. Define a small message format.

For your inventory app, JSON Lines is simple and good:

```
{"msg_id":"m001","type":"ADD_STOCK","barcode":"6921234567890","qty":5,"source":"supplier","timestamp":"2026-05-03T18:20:00Z"}
```

Each message should end with newline:

```
\n
```

So Qt knows where one message ends.

Example stream:


```
{ "msg_id": "m001", "type": "ADD_STOCK", "barcode": "6921234567890", "qty": 5 }
```

```
{ "msg_id": "m002", "type": "REMOVE_STOCK", "barcode": "6921234567890", "qty": 2, "destination": "customer" }
```

6. ACK response

Qt should reply after successfully receiving and saving data.

Example success response:

```
{ "msg_id": "m001", "status": "ACK" }
```

Example failure response:

```
{ "msg_id": "m001", "status": "NACK", "reason": "UNKNOWN_BARCODE" }
```

This is important because your mobile/scanner side should only reset the newly added quantity to zero after Qt confirms success.

Recommended rule:

```
Mobile/scanner sends stock update
```

```
Qt validates and saves it
```

```
Qt sends ACK
```

```
Only after ACK, sender clears local pending quantity
```

7. Data types you should support

For your current inventory system, prepare these message types:

```
ADD_STOCK
```

```
REMOVE_STOCK
```

UNKNOWN_BARCODE_FOUND

PRODUCT_INFO_UPDATED

SYNC_REQUEST

SYNC_ACK

PING

PONG

Minimum MVP messages:

ADD_STOCK

REMOVE_STOCK

UNKNOWN_BARCODE_FOUND

ACK

NACK

8. Qt database side

Qt should be ready to store incoming data.

Minimum tables or models:

products

stock_movements

unknown_barcodes

sync_messages

`sync_messages` is important for duplicate protection.

Example fields:

`msg_id`

`device_id`

`message_type`

`payload`

`status`

`received_at`

`processed_at`

This prevents duplicate updates when the sender retries the same message.

9. Error handling needed

Prepare for these cases:

Problem	Required behavior
Bluetooth disconnected	Keep local pending messages
Qt receives duplicate <code>msg_id</code>	Return ACK again, do not apply twice
Unknown barcode	Add to Unknown List
Invalid JSON	Return NACK
Qt database save failed	Return NACK
No ACK received	Sender retries later

Partial message received

Qt waits until newline

10. Recommended MVP setup

For the first working version, keep it very simple:

HC-05 baud rate: 115200

Message format: JSON line

Connection direction: Qt connects to HC-05

Qt transport: QSerialPort if Windows creates COM port

ACK required: yes

Image transfer: no

The MVP flow:

1. User scans barcode on mobile/scanner side
2. Sender creates JSON message
3. Sender sends message through HC-05
4. Qt receives one JSON line
5. Qt saves stock movement
6. Qt replies ACK
7. Sender clears pending quantity

My recommendation: for Qt desktop, start with QSerialPort, not direct Qt Bluetooth. It is usually easier because HC-05 behaves like a normal serial COM port after pairing.

I am going to use STM32 series chip. which one is best?

I'll compare this for your real use case: STM32 + HC-05 + barcode/stock messages, not a generic "best MCU" answer. Main things are UART reliability, power, flash/RAM, USB/debug ease, and future expansion.

Thought for 6s

For your STM32 + HC-05 + inventory/barcode sync device, the best practical choice is:

Best choice: STM32G0B1

Specifically:

```
STM32G0B1CBT6
```

or development board:

```
NUCLEO-G0B1RE
```

Why STM32G0B1 is best for your case

Your device does not need heavy CPU power. It mainly needs:

```
barcode input
```

```
UART communication with HC-05
```

```
small message buffer
```

```
local pending-message storage
```

```
ACK/NACK retry logic
```

```
maybe USB debug/configuration
```

```
low cost
```

stable production availability

STM32G0B1 fits this very well. The STM32G0 family is a modern Cortex-M0+ line, and ST lists G0 parts with UART/USART interfaces, low-voltage operation, and practical flash/RAM sizes for embedded communication devices. The G0B1 line also has much more RAM/flash than tiny G030/G031 parts, which gives you breathing room for JSON parsing, queues, retries, and future features.

My recommended ranking

Rank	STM32 chip	Best for	My opinion
1	STM32G0B1CBT6	Production barcode + HC-05 device	Best balance
2	STM32G431CBT6	Future advanced device	More powerful than needed
3	STM32F103C8T6	Cheap prototype / Blue Pill	Okay, but older
4	STM32G030 / G031	Very cheap small device	Too tight for comfortable development
5	STM32F4 series	High-performance embedded system	Overkill

Simple recommendation

Use this for development:

```
NUCLEO-G0B1RE
```

Use this for final PCB:

```
STM32G0B1CBT6
```

That gives you enough UARTs, memory, and future expansion without making the hardware expensive or complicated.

Why not STM32F103C8?

STM32F103C8 is popular and cheap, and ST still lists it as active/in volume production with 64 KB flash, 72 MHz Cortex-M3 CPU, USB, and CAN.

But I would not choose it as the first option for a new product because:

It is older.

Many cheap Blue Pill boards use clone chips.

USB/debug setup can be annoying.

The ecosystem is huge, but quality varies.

It is fine for learning. For your real app, I would rather start with STM32G0 or STM32G4.

Why not STM32G431?

STM32G431 is excellent. It is a Cortex-M4 MCU running up to 170 MHz, with DSP/FPU features.

But for HC-05 communication, barcode scanning, and stock message syncing, it is more powerful than you need.

Choose STM32G431 only if you later want:

motor control

advanced sensors

fast signal processing

more complex UI/device logic

more real-time control

For your inventory sync device, G431 is nice but a little “buying a sports car to deliver one pizza.”

Minimum hardware features you should require

For your board/chip, make sure it has:

At least 2 UART/USART ports

At least 64 KB flash

At least 16 KB RAM

SWD debug pins

3.3V operation

USB support if you want easy PC config/debug

Enough GPIO for scanner/button/LED/buzzer

Why 2 UARTs?

UART1: HC-05 Bluetooth

UART2: barcode scanner module or debug serial

If your barcode scanner connects by USB instead of UART, then the design changes. But for simple embedded scanner modules, UART scanner + UART HC-05 is very clean.

Recommended architecture

Barcode scanner

↓ UART

STM32G0B1

↓ UART

HC-05

↓ Bluetooth SPP

Qt desktop app

Inside STM32 firmware:

Read barcode

Build message

Send to HC-05

Wait for ACK from Qt

If ACK received → clear pending item

If no ACK → keep and retry

Final answer

Use:

STM32G0B1CBT6

For development:

NUCLEO-G0B1RE

For a very cheap prototype only:

STM32F103C8T6 / Blue Pill

But for your actual product direction, I would choose STM32G0B1. It is modern, not overkill, and strong enough for reliable HC-05 barcode stock syncing.

is it cheaper than STM32F407?

Thought for 5s

Yes — STM32G0B1 is much cheaper than STM32F407.

Chip	Example price, 1 pc	Example price, 100 pcs	Notes
STM32G0B1CBT6	~\$3.99–\$4.15	~\$2.51–\$2.60	Good for HC-05 + barcode sync
STM32F407VGT6	~\$11.87–\$12.34	~\$8.33–\$8.51	Much more powerful, but overkill

Mouser lists STM32G0B1CBT6 at \$3.99 for 1 pc and \$2.51 at 100 pcs. DigiKey lists it at \$4.15 for 1 pc and \$2.604 at 100 pcs.

For STM32F407VGT6, Mouser shows around \$11.87 for 1 pc, and DigiKey shows \$12.34 for 1 pc and \$8.33489 at 90 pcs.

So roughly:

STM32G0B1 = about 3x to 4x cheaper than STM32F407

For your HC-05 communication + barcode + stock sync device, STM32F407 is unnecessary unless you need camera processing, display UI, heavy signal processing, or many peripherals. For this project, STM32G0B1 is the better cost choice.

give me the barcode scanning speed in mobile

Thought for a couple of seconds

Mobile barcode scanning speed is usually fast enough for inventory work, but the real speed depends more on the camera, lighting, barcode quality, and app design than raw phone performance.

Typical mobile barcode scanning speed

Situation	Typical scan speed
Good lighting, clear barcode	0.2–0.5 seconds per scan
Normal warehouse/store condition	0.5–1.5 seconds per scan
Poor lighting / blurry barcode	2–5 seconds or failed scan
Continuous scanning mode	Around 30–120 scans/minute
Careful manual scanning	Around 20–60 scans/minute

So for your app, a realistic assumption is:

- Safe design speed: 1 barcode per second
- Good condition speed: 2-3 barcodes per second
- Manual real-world speed: 30-80 products per minute

What controls scan speed

The biggest factors are:

- Camera autofocus speed
- Barcode size and print quality
- Lighting condition
- Distance from barcode

Whether user must tap after each scan

Whether the app supports continuous scan mode

Duplicate detection delay

Sound/vibration feedback speed

For your inventory app

For adding/removing stock, I recommend continuous scan mode:

1. User opens Add Stock or Remove Stock
2. Camera stays open
3. App detects barcode automatically
4. App vibrates/beeps
5. Quantity increments immediately
6. App ignores the same barcode for a short cooldown

Example cooldown:

Same barcode cooldown: 700ms-1200ms

Different barcode cooldown: 0ms-300ms

This prevents accidental repeated scans when the user keeps the camera on the same barcode.

Recommended UX behavior

For your case, this is best:

Scan barcode once → qty +1

User can manually enter repeated number → qty +N

Unknown barcode → store barcode + optional local photo only in Unknown List

Sync to Qt later via Bluetooth

Example:

Barcode: 6921234567890

Scanned count: 1

User enters repeated number: 24

Final quantity to add: 24

That is much faster than scanning the same product 24 times.

Practical expectation

For a normal mobile camera scanner:

1 product type with quantity entry: very fast

Many different product types: 30-80 scans/minute

Same barcode repeated many times: use repeated quantity, not repeated scanning

So yes, mobile barcode scanning is fast enough. The bigger UX win is your idea: scan once, then enter quantity manually.